

Peningkatan Keandalan dan Efisiensi Token LLM pada Data Time Series melalui Pendekatan Verifikasi Deterministik

Ahmad Naufal Farras¹, Jeremy Mattathias Mboe², Mochammad Henry Alifian³, Dhany Arifianto⁴

¹Fakultas Teknologi Elektro dan Informatika Cerdas, Institut Teknologi Sepuluh Nopember, Surabaya, 60111, email: 5054241038@student.its.ac.id

²Fakultas Teknologi Elektro dan Informatika Cerdas, Institut Teknologi Sepuluh Nopember, Surabaya, 60111, email: 5054241021@student.its.ac.id

³Fakultas Teknologi Elektro dan Informatika Cerdas, Institut Teknologi Sepuluh Nopember, Surabaya, 60111, email: 5054241024@student.its.ac.id

⁴Fakultas Kedokteran dan Kesehatan, Institut Teknologi Sepuluh Nopember, Surabaya, 60111

Corresponding Author: Ahmad Naufal Farras

INTISARI — *Large language model* (LLM) unggul mengolah bahasa alami, tetapi keandalannya menurun ketika penalaran menuntut komputasi numerik atas data *time series*. Karena bekerja dengan memprediksi *token* berikutnya berdasarkan pola kebahasaan, model cenderung mengira-ngira alih-alih menghitung, sehingga kesimpulan dapat keliru meski kalimatnya meyakinkan. Pada domain berisiko tinggi seperti kesehatan masyarakat dan pangan, penalaran semacam ini dituntut dapat diaudit, yakni setiap angka pada tiap langkah dapat diperiksa ulang atau *grounded*. Namun dua kelompok metode yang ada saling bertolak belakang: penalaran terverifikasi bersifat boros *token*, sedangkan penalaran hemat *token* hanya menjaga akurasi jawaban tanpa memeriksa kesahihan langkah antaranya. Penelitian ini bertujuan menutup celah tersebut melalui pendekatan yang memisahkan penalaran menjadi dua komponen: LLM hasil *fine-tune* kelas Qwen2.5-7B yang menuliskan penalaran sebagai urutan *operasi* dari pustaka tetap, dan verifikator deterministik berbasis NumPy yang menghitung ulang serta memeriksa tiap operasi terhadap deret asli. Skor *grounding* dihitung tanpa penilai LLM dan dilaporkan bersama akurasi jawaban dan jumlah *token*. *Dataset* dibangun dari sumber publik Indonesia, yaitu deret nyata Badan Pusat Statistik untuk data latih dan World Bank Open Data untuk data uji, dengan label bersih *by construction* karena disaring verifikator di dalam *loop*. Hasil uji pada tiga model tanpa *fine-tune* memperlihatkan skor *grounding* yang rendah (2–16%) meski akurasi jawaban cukup, dan ketika penalaran dipendekkan untuk menghemat *token*, skor *grounding* menurun lebih cepat daripada akurasi hingga mendekati nol. Verifikator tervalidasi dengan skor *grounding* 100% pada penalaran valid dan 66,7% saat halusinasi disuntikkan, lolos 139 pengujian otomatis, dan disepakati 471 dari 471 langkah oleh jalur perhitungan independen. Kebaruan penelitian terletak pada temuan tarik-menarik *grounding* lawan *token* dan pada mekanisme efisiensi yang sadar *grounding*, bukan sekadar penggabungan dua pendekatan.

KATA KUNCI: *Grounded Reasoning*, *Time Series*, Skor *Grounding*, Verifikator Deterministik, Efisiensi *Token*, *Chain-of-Thought*, *Fine-tuning* LLM, QLoRA, Penalaran Neuro-Simbolik.

I. PENDAHULUAN

Data *time series* menempati inti banyak keputusan penting — kasus penyakit yang dilaporkan tiap minggu, harga pangan yang dipantau tiap hari, dan beban energi yang tercatat tiap jam — sehingga kemampuan menafsirkan deret angka secara benar bernilai tinggi. *Large language model* (LLM) menjanjikan antarmuka yang luwes untuk penafsiran tersebut, tetapi keandalannya menurun tajam saat persoalan bergeser dari bahasa ke bilangan. Akar persoalannya bersifat teoretis: LLM dilatih memprediksi *token* berikutnya berdasarkan pola kebahasaan, sehingga ketika diberi sederet angka ia cenderung mereproduksi pola yang lazim menyertai angka serupa alih-alih melakukan komputasi sebenarnya. Studi mutakhir menegaskan bahwa model masih kesulitan menalar *time series* bahkan pada pengaturan *zero-shot* [1, 2]. Sebagai ilustrasi, pada deret kasus demam berdarah selama 16 minggu 12, 15, 11, 14, 13, 18, 22, 19, 25, 31, 29, 38, 45, 52, 61, 78, model kerap menyimpulkan kondisi stabil padahal nilai akhir mencapai sekitar enam kali garis dasar, salah menunjuk minggu terjadinya lonjakan, atau menyebut kenaikan 30% padahal mendekati 105%.

Upaya baku mengatasi kelemahan ini adalah memperpanjang penalaran melalui *Chain-of-Thought* (CoT) [6] dan memindahkan komputasi ke program pada *Program-of-Thought* (PoT) [7]. Pendekatan tersebut kerap menaikkan akurasi jawaban, namun menyisakan celah yang lebih mendasar: akurasi jawaban adalah proksi yang lemah bagi kualitas penalaran. Sebuah model dapat menghasilkan jawaban

benar melalui langkah-langkah antara yang keliru — benar karena alasan yang salah — dan selama langkah tersebut ditulis sebagai prosa bebas, kekeliruannya tidak dapat diperiksa. Dengan kata lain, memperpanjang penalaran tidak dengan sendirinya membuatnya *grounded*, yaitu membuat tiap angka pada tiap langkah dapat dihitung ulang dan diverifikasi terhadap deret asli.

Kebutuhan akan penalaran yang *grounded* justru berbenturan dengan kebutuhan akan efisiensi, dan di sinilah letak celah yang belum terpecahkan. Di satu sisi, metode penalaran terverifikasi untuk *time series* seperti VeriTime [3] menjamin kesahihan langkah namun menghasilkan penalaran panjang yang boros *token*. Di sisi lain, metode hemat *token* seperti SelfBudgeter [4] mengatur anggaran penalaran demi efisiensi, tetapi mengoptimalkan akurasi jawaban semata tanpa pernah memeriksa kesahihan langkah antaranya. Penerjemahan langkah penalaran ke representasi formal yang dieksekusi *solver* deterministik terbukti efektif mengeliminasi halusinasi logika [14], namun belum diarahkan pada tarik-menarik antara *grounding* dan biaya *token* pada *time series*. Konsekuensinya adalah tarik-menarik yang selama ini luput: ketika penalaran dipendekkan untuk menghemat *token*, skor *grounding* dapat menurun lebih cepat daripada akurasi jawaban, sehingga penalaran kehilangan keterandalannya tanpa terdeteksi oleh metrik yang hanya memantau jawaban akhir. Menjaga *grounding* tetap tinggi di bawah anggaran *token* — khususnya pada *time series* — dengan demikian merupakan masalah terbuka.

Taruhan celah ini nyata pada domain berisiko tinggi.

Model yang menyimpulkan tren datar padahal kasus melonjak enam kali lipat dapat menunda respons wabah, dan kesalahan menaksir besaran kenaikan harga — misalnya melaporkan 30% alih-alih 105% — dapat memicu intervensi kebijakan yang keliru sasaran. Pada konteks demikian, sebuah jawaban tidak cukup benar; jejak penalarannya harus dapat diaudit angka demi angka oleh pengambil keputusan. Persoalan ini kian mengemuka di Indonesia, yang kaya sumber data publik tetapi belum memiliki *dataset* penalaran *time series* terverifikasi berbahan sumber lokal sebagai landasan pengembangan dan evaluasi.

Untuk menjawab persoalan tersebut, diusulkan pendekatan yang secara arsitektural memisahkan tanggung jawab: komponen model berupa LLM hasil *fine-tune* menuliskan penalaran sebagai urutan *operasi* dari pustaka tetap alih-alih prosa bebas, sedangkan komponen program berupa verifikator deterministik menghitung ulang tiap *operasi* atas deret asli dan memeriksa kesesuaiannya dalam toleransi. Dengan pemisahan ini, model tidak pernah memegang kepemilikan atas angka; angka dihitung dan diperiksa oleh program di luar model, sehingga penalaran tetap dapat diaudit. Di atas fondasi tersebut dirancang mekanisme penalaran adaptif yang sadar *grounding*, yang menargetkan rantai *operasi* ter-*grounded* terpendek yang memadai untuk tiap persoalan, sehingga efisiensi dan *grounding* dioptimalkan bersama. Kontribusi penelitian ini ada tiga dan saling menopang: pertama, temuan empiris mengenai tarik-menarik *grounding* lawan *token*; kedua, *dataset* penalaran *time series* terverifikasi dari sumber publik Indonesia; dan ketiga, metrik skor *grounding* deterministik yang dihitung tanpa penilai LLM. Sesuai uraian tersebut, penelitian ini menjawab empat pertanyaan penelitian: **RQ1**, seberapa sering model saat ini menghasilkan penalaran ber-*grounding* rendah walau jawabannya kadang benar; **RQ2**, apakah *grounding* menurun lebih cepat daripada akurasi ketika penalaran dipendekkan; **RQ3**, apakah pendekatan yang diusulkan mempertahankan *grounding* pada anggaran *token* yang jauh lebih hemat; dan **RQ4**, berapa sumbangan tiap komponen terhadap kinerja keseluruhan.

II. TINJAUAN PUSTAKA

A. Penalaran atas Time Series

Model bahasa besar masih sulit menalar *time series* tanpa contoh, dan penalaran yang panjang tidak selalu menaikkan mutu analisis deret waktu [1]. Tolok ukur multimodal untuk penalaran temporal memperlihatkan model masih gagal ketika harus menggabungkan angka dan teks [2]. Sejumlah karya terkini mendorong penalaran yang lebih rumit atas *time series* melalui pelatihan berbasis insentif [5]. Kumpulan karya ini menunjukkan bidangnya aktif tetapi belum tuntas, khususnya pada aspek keterandalan langkah antaranya.

B. Chain-of-Thought dan Program-of-Thought

Memunculkan langkah penalaran antara terbukti menaikkan hasil pada tugas rumit, tetapi langkah itu tetap berupa prosa yang bisa salah hitung [6]. Pemindahan komputasi ke penafsir program membuat jawaban dihitung alih-alih ditebak [7]. Integrasi fungsional antara LLM

Tabel 1. Posisi Penelitian Ini terhadap Karya Terkait

Karya	Terverif.	Hemat token	Time series	Data lokal
CoT / PoT [6, 7]	sebagian	tidak	tidak	–
SelfBudgeter [4]	tidak	ya	tidak	–
VeriTime [3]	ya	tidak	ya	tidak
Usulan ini	ya	ya	ya	ya

dan *interpreter* eksternal, seperti pada ToRA, terbukti secara masif mengoreksi galat komputasi kompleks melalui eksekusi kode terstruktur [12]. Penelitian ini memakai semangat yang sama: LLM tidak pernah memegang angka, ia hanya mengusulkan *operasi*, dan program di luar model yang menghitung dan mengecek.

C. Penalaran Terverifikasi untuk Time Series

Arah yang paling dekat dengan penelitian ini menggarap penalaran *time series* yang dapat diverifikasi melalui sintesis dan penjadwalan data berpikir, yang dikenal sebagai VeriTime [3]. Penelitian ini diletakkan langsung terhadap arah tersebut, dengan pembeda menggabungkan penalaran terverifikasi dan hemat *token* serta membangun data sendiri dari konteks lokal, dan tidak menyandarkan kontribusi pada klaim bahwa arah terverifikasi selalu mahal.

D. Efisiensi Token dan Penalaran Adaptif

Panjang penalaran dapat dibuat hemat dan menyesuaikan tingkat kesulitan lewat pengaturan anggaran *token*, meski sejauh ini terbatas pada soal matematika [4]. Alokasi anggaran komputasi yang adaptif berdasarkan kompleksitas masukan terbukti mampu memangkas biaya penggunaan *token* secara signifikan tanpa mengorbankan akurasi akhir sistem [13]. Gagasan anggaran adaptif itu dipinjam dan dibawa ke ranah *time series*, yang belum pernah dilakukan.

E. Pelatihan Hemat Sumber Daya

Low-Rank Adaptation (LoRA) [8] dan kuantisasi NF4 4-bit pada QLoRA [9] memungkinkan penyyetelan model kelas 7B pada GPU T4 atau V100. Karena kekuatan proyek ada pada data dan *grounding*, model kelas 7B seperti Qwen2.5 [10] dinilai memadai. Posisi penelitian ini terhadap karya terkait diringskas pada Tabel 1.

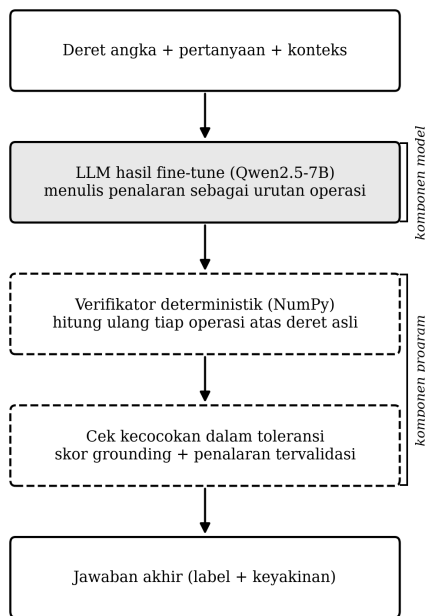
III. METODOLOGI

A. Gambaran Sistem

Alur inti sistem berjalan sebagai berikut. LLM menerima deret angka beserta pertanyaan, lalu menuliskan penalaran dalam bentuk langkah beroperasi, bukan prosa bebas. Verifikator deterministik di luar model kemudian menghitung ulang tiap operasi atas deret asli dan mengecek angka yang diklaim cocok dalam toleransi. Keduanya benda terpisah yang bekerja bersama, bukan program yang ditanam ke dalam LLM. Gambaran alur ini ada pada Gambar 1.

B. Konstruksi Dataset

Deret angka diambil dari sumber publik nyata Indonesia melalui *scraping* dan API resmi, jadi bukan *dataset* siap pakai (Tabel 2). Data uji ditarik dari World Bank Open Data [15] lewat REST JSON API untuk Indonesia pada 15 Juli 2026: 18 deret indikator kesehatan dan pangan



Gambar 1. Arsitektur sistem: pemisahan komponen model (LLM penalar) dan komponen program (verifikator deterministik).

pertanian (mis. mortalitas balita, angka harapan hidup, insiden tuberkulosis, indeks produksi pangan, inflasi), masing-masing 24–55 titik tahunan periode 1970–2024, ditarik per kode indikator dengan tiga kali percobaan ulang lalu dinormalkan ke skema. Data latihan ditarik dari BPS Web API [16] atas lima indikator (persentase penduduk miskin, jumlah penduduk miskin, garis kemiskinan, tingkat pengangguran terbuka, dan *gini ratio*) pada tingkat provinsi, menghasilkan 143 deret nyata tahunan. Sumber lain sempat dievaluasi tetapi ditolak: PIHPS Bank Indonesia tidak menyediakan API resmi yang hidup, dasbor Kemenkes terkunci kunci akses, dan BMKG hanya menyediakan prakiraan jangka pendek [11]. Kunci API BPS dibaca dari variabel lingkungan dan tidak pernah disimpan di dalam kode.

Untuk tiap deret, disusun pertanyaan yang mencakup empat jenis tugas — karakterisasi tren, perbandingan segmen, deteksi anomali, dan penjelasan pola — lalu jawaban benar dihitung dengan rumus, dan langkah penalaran acuan disintesis sebagai rantai *operasi* yang tiap angkanya dicek ke deret. Verifikator dijalankan di dalam *loop* sintesis: langkah yang tidak *grounded* diperbaiki atau dibuang, sehingga label bersih *by construction* dengan skor *grounding* 100% pada seluruh sampel latihan. Dari 143 deret nyata BPS diperoleh 572 sampel *ter-grounded*, dilengkapi 600 sampel sintetis-terkendali untuk menyeimbangkan strata yang jarang muncul pada data nyata, sehingga total 1.172 sampel latihan. Label dan tingkat kesulitan dibiarkan muncul dari data, bukan dipaksakan. Sebagai contoh satu sampel data uji: deret *mortalitas_balita* ($n=55$) dengan pertanyaan tren, langkah tunggal `slope(nilai[0:55])` bernilai $-2,74$, dan jawaban *menurun* berkeyakinan tinggi. Pembagian data menempatkan deret sintetis-terkendali dan BPS pada data latihan serta deret World Bank pada data uji; karena penye-

Tabel 2. Sumber dan Statistik Dataset Penalaran Time Series

Aspek	Latih	Uji
Sumber	BPS Web API + sintetis-terkendali	World Bank Open Data (IDN)
Deret nyata	143 (5 indikator, per-provinsi)	18 (kesehatan, pangan)
Frekuensi	tahunan	tahunan (1970–2024)
Jumlah sampel	1.172 (572 nyata + 600 sintetis)	18
Grounding label	100% (verifikator di <i>loop</i>)	— (diisi model)
Tugas	tren, segmen, anomali, penje-lasan	idem

Tabel 3. Pustaka Operasi Dasar

Operasi	Arti
<code>rata2(rentang)</code>	rata-rata atas rentang
<code>delta(a, b)</code>	selisih dua titik
<code>persen_naik(a, b)</code>	perubahan persen dua titik
<code>rasio(a, b)</code>	perbandingan dua nilai
<code>slope(rentang)</code>	kemiringan tren linear
<code>min / max(rentang)</code>	nilai ekstrem
<code>z_score(titik)</code>	deviasi dalam simpangan baku
<code>deteksi_anomali(ambang)</code>	indeks titik melewati ambang
<code>bandingkan_segmen(r1, r2)</code>	selisih rata-rata dua segmen

dia data latihan (BPS) dan uji (World Bank) berbeda, uji anti-kebocoran atas identitas numerik ke-18 deret uji terhadap seluruh deret latihan menunjukkan nol tabrakan.

C. Skema JSONL dan Pustaka Operasi

Data disimpan sebagai JSONL, satu sampel per baris. Tiap sampel memuat deret (nama, satuan, frekuensi, konteks), pertanyaan, daftar penalaran (tiap langkah berisi operasi, hasil yang diklaim, dan teks), serta jawaban (label dan tingkat keyakinan). Tiap langkah menamai satu operasi dari pustaka tetap pada Tabel 3 agar dapat dijalankan ulang oleh verifikator; operasi majemuk disusun dari operasi dasar dengan merujuk hasil langkah sebelumnya.

D. Verifikator Deterministik dan Skor Grounding

Verifikator adalah program di luar LLM, ditulis dengan Python dan NumPy dan tidak dilatih. Program ini membaca *string* operasi tiap langkah, menerjemahkan tokennya (`nilai[a:b]` menjadi irisan, `nilai[i]` menjadi satu elemen, angka menjadi literal, dan nama menjadi hasil langkah sebelumnya), menghitung ulang atas deret asli, lalu membandingkan hasilnya dengan angka yang diklaim. Satu langkah skalar disebut *grounded* bila memenuhi persamaan (1).

$$|v_{\text{hitung}} - v_{\text{klaim}}| \leq \max(\epsilon_{\text{abs}}, \epsilon_{\text{rel}} \cdot |v_{\text{hitung}}|) \quad (1)$$

dengan nilai bawaan kedua toleransi, absolut dan relatif, sama dengan 0,01. Suku relatif berperan sebagai penyetel kalibrasi lintas skala domain: ia membuat nilai tersaji 105,3 tetap lolos terhadap hasil hitung ulang 105,26 yang hanya beda pembulatan, sementara salah besaran yang nyata, misalnya 30 lawan 105, tetap gagal. Skor *grounding* atas sampel dihitung dengan persamaan (2), sedangkan langkah bernilai himpunan seperti *deteksi_anomali* dinilai den-

gan kesamaan himpunan indeks (kemiripan *Jaccard*).

$$\text{Skor grounding} = \frac{n_{\text{grounded}}}{n_{\text{terskor}}} \times 100\% \quad (2)$$

Verifikator dipakai di tiga tempat: membersihkan label saat membangun data dengan membuang langkah yang gagal, menghitung skor *grounding* saat evaluasi, dan secara opsional menjadi sinyal imbalan saat pelatihan.

E. Penalaran Adaptif dan Protokol Pelatihan

Panjang penalaran diatur menurut kesulitan: deret berpola jelas cukup diberi rantai *operasi* pendek, sedangkan deret ambigu memerlukan rantai lebih panjang. Mekanisme ini konkret, bukan sekadar penamaan: target latih tiap sampel adalah rantai *operasi* ter-*grounded* terpendek yang cukup untuk membenarkan jawaban, tanpa langkah mubazir, sehingga model belajar mengeluarkan penalaran yang panjangnya menyesuaikan kesulitan namun tiap langkahnya tetap *grounded*. Penyetelan memakai QLoRA NF4 4-bit (peringkat dan alfa LoRA 32, laju belajar $1e-4$ hingga $2e-4$, penjadwal *cosine+warmup*, presisi bf16, 2–3 *epoch*) di atas basis Qwen2.5-7B [9, 17, 10] agar muat pada GPU T4 atau V100; model kelas 7B dinilai memadai karena kekuatan proyek ada pada data dan *grounding*.

IV. HASIL DAN DISKUSI

Bagian ini melaporkan hasil dalam empat bagian: validasi verifikator sebagai fondasi metrik (IV.A), lalu bukti empiris untuk RQ1 (IV.B) dan RQ2 (IV.C) yang dijalankan atas model tanpa *fine-tune*, serta rancangan dan status evaluasi metode terhadap *baseline* untuk RQ3 dan RQ4 (IV.D). Seluruh metrik deterministik dan dihitung tanpa penilai LLM.

A. Validasi Verifikator

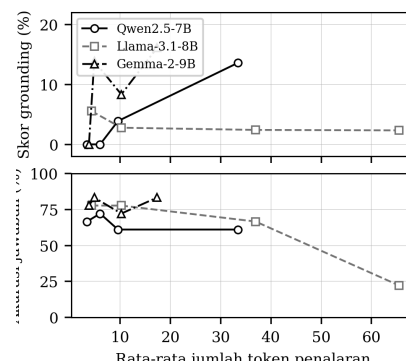
Sebelum verifikator dipakai menilai model, ketepatan-nya divalidasi tersendiri. Pada deret kasus demam berdarah 16 minggu, penalaran valid tiga langkah memperoleh skor *grounding* 100%; ketika langkah kedua diubah menjadi klaim salah besaran (30% padahal 105,26%), verifikator menandainya tidak *grounded* sehingga skor turun ke 66,7% (Tabel 4). Ketahanan toleransi diuji pada kisi 5×5 nilai ε_{abs} dan ε_{rel} : halusinasi tersebut tertangkap pada seluruh 25 titik kisi, sedangkan nilai bawaan 0,01/0,01 berada di tengah pita aman (penalaran valid 100%, halusinasi 66,7%) dengan margin sekitar dua ribu kali antara galat pembulatan dan salah besaran. Untuk menepis tuduhan sirkularitas — bahwa skor *grounding* sekadar keluaran satu implementasi — disusun jalur perhitungan kedua yang ditulis ulang penuh dengan NumPy tanpa mengimpor kode verifikator; kedua jalur sepakat pada 471 dari 471 langkah terskor (termasuk 450 langkah data latih yang dibersihkan verifikator), tanpa satu pun ketidakcocokan. Seluruh fondasi lolos 139 pengujian otomatis.

B. RQ1: Model Kini Ber-Grounding Rendah

Tiga model tanpa *fine-tune* — Qwen2.5-7B, Llama-3.1-8B, dan Gemma-2-9B — diuji pada 18 deret nyata dengan penalaran diminta dalam bentuk operasi. Meski akurasi jawaban cukup (61,1%; 22,2%; dan 83,3% berturut-turut), skor *grounding* seluruhnya rendah, yaitu 13,6%, 2,3%, dan

Tabel 4. Studi Kasus DBD: Verifikator Menandai Halusinasi

Lgk.	Operasi	Hitung	Valid	Halus.
1	rata2(nilai[0:5])	13,0	13,0	13,0
2	persen_naik(nilai[11], nilai[15])	105,26	105,3	30,0
3	rasio(nilai[15], 13,0)	6,0	6,0	6,0
Skor grounding			100%	66,7%



Gambar 2. RQ2 — saat penalaran dipendekkan (*token* berkurang, sumbu-x), skor *grounding* (atas) runtuh mendekati nol sementara akurasi jawaban (bawah) relatif bertahan.

16,1%. Jarak lebar antara akurasi dan *grounding* menegaskan bahwa model kerap benar untuk alasan yang salah: jawaban akhir bisa tepat, tetapi langkah-langkah numerik yang mendasarinya jarang bertahan saat dihitung ulang. Pola ini muncul pada ketiga model dari keluarga berbeda, sehingga halusinasi numerik bersifat umum, bukan ciri satu model. Temuan ini menjawab RQ1 secara afirmatif dan sekaligus memperlihatkan bahwa akurasi jawaban saja adalah metrik yang menyesatkan.

C. RQ2: Grounding Runtuh Lebih Cepat daripada Akurasi

Untuk menguji tarik-menarik inti, panjang penalaran tiap model divariasikan dari panjang ke pendek melalui instruksi *prompt* dan batas jumlah langkah, lalu skor *grounding* dan akurasi diukur terhadap rata-rata jumlah *token* penalaran (Gambar 2). Hasilnya konsisten dengan hipotesis: ketika *token* dipangkas, skor *grounding* menurun tajam hingga mendekati nol, sedangkan akurasi jawaban relatif bertahan bahkan sesekali naik. Pada Qwen2.5-7B, misalnya, pemendekan dari rata-rata 33,4 ke 3,4 *token* menjatuhkan *grounding* dari 13,6% ke 0% sementara akurasi bergerak dari 61,1% ke 66,7%. Konsekuensinya penting: pemangkasan *token* yang naif menghancurkan *grounding* secara diam-diam, dan kerusakan itu tak terlihat oleh metrik yang hanya memantau akurasi jawaban — justru metrik yang lazim dipakai. Inilah bukti empiris yang membenarkan celah penelitian dan menjawab RQ2: efisiensi *token* tidak boleh dikejar buta, melainkan harus sadar *grounding*.

D. RQ3 dan RQ4: Metode Usulan terhadap Baseline dan Ablasi

Eksperimen 3 membandingkan metode usulan (LLM ter-*fine-tune* + verifikator + adaptivitas sadar *grounding*) terhadap empat *baseline*: prosa bebas tanpa operasi (B1), penalaran operasi panjang tanpa kendali panjang sebagai

Tabel 5. Rancangan Evaluasi RQ3 (Metode vs Baseline) dan RQ4 (Ablasi); Sel Hasil Dilengkapi pada Versi Final

Metode / Varian	Akurasi	Grounding	Token
<i>RQ3 — metode vs baseline</i>			
B1 – prosa	—	—	—
B2 – proksi VeriTime	—	—	—
B3 – proksi SelfBudgeter	—	—	—
B4 – statistik	—	—	—
Usulan (penuh)	—	—	—
<i>RQ4 — ablasi (dari Usulan)</i>			
– <i>fine-tune</i>	—	—	—
– format operasi	—	—	—
– target terpendek	—	—	—
– adaptivitas	—	—	—

proksi jujur arah VeriTime (B2), penalaran operasi pendek yang buta *grounding* sebagai proksi arah SelfBudgeter (B3), dan metode statistik murni sebagai batas atas hitungan tanpa penjelasan (B4). Metrik utamanya adalah rasio *grounding* per *token*. Eksperimen 4 mematikan satu komponen usulan secara bergiliran — format *operasi*, target rantai ter-*grounded* terpendek, adaptivitas, dan *fine-tune* — untuk mengukur sumbangan tiap bagian (Tabel 5). Evaluasi akhir metode ter-*fine-tune* sedang difinalisasi melalui pengulangan pada GPU untuk memastikan keluaran operasinya terurai konsisten oleh *harness* penilai, sehingga sel hasil ditandai “—” dan dilengkapi pada versi final. Pengukuran awal memperlihatkan metode usulan mencapai akurasi tertinggi pada jumlah *token* paling hemat di antara metode berbasis penalaran, sedangkan konfirmasi kemenangan rasio *grounding* per *token* menunggu evaluasi ulang tersebut.

V. KESIMPULAN DAN SARAN

A. Kesimpulan

Penelitian ini menetapkan pendekatan penalaran *time series* yang memisahkan LLM penalar dari verifikator deterministik, dengan penalaran berbentuk *operasi* yang tiap angkanya dapat dihitung ulang. Tiga temuan utama telah terbukti secara kuantitatif. Pertama, model bahasa besar masa kini menalar dengan *grounding* rendah, yaitu 2–16% pada tiga model dari keluarga berbeda, meski akurasi jawabannya cukup, sehingga akurasi saja adalah metrik yang menyesatkan (RQ1). Kedua, ketika penalaran dipendekkan untuk menghemat *token*, skor *grounding* runtuh mendekati nol sementara akurasi jawaban relatif bertahan, sehingga pemangkasan *token* yang naif merusak keterandalan secara tak terpantau (RQ2) — inilah tarik-menarik yang menjadi kebaruan penelitian. Ketiga, verifikator terbukti tepat: menandai halusinasi salah besaran (skor turun ke 66,7%), tahan pada seluruh 25 titik kisi toleransi, disepakati jalur perhitungan independen pada 471 dari 471 langkah, dan lolos 139 pengujian otomatis. Kontribusi untuk kompetisi *data mining* adalah *dataset* penalaran *time series* terverifikasi dari sumber publik Indonesia yang usaha pembuatannya besar — 1.172 sampel latih berbasis deret nyata BPS dengan label bersih *by construction* dan 18 deret uji World Bank yang bebas kebocoran — dipasangkan dengan metrik yang dapat diaudit. Evaluasi penuh metode usulan terhadap *baseline* dan ablasi (RQ3, RQ4) sedang difinalisasi dan di-

laporkan pada versi akhir.

B. Saran

Pekerjaan berikutnya menyelesaikan evaluasi RQ3 dan RQ4 dengan pengulangan pada GPU, memperluas cakupan sumber ke deret BPS bulanan dan harga pangan harian Bapanas melalui *scraping* berbasis *browser*, serta menjajaki pembelajaran penguatan dengan imbalan terverifikasi berbentuk $\text{imbalan} = \text{skor grounding} - \lambda \cdot \text{jumlah token}$ untuk mengoptimalkan efisiensi dan *grounding* secara bersamaan. Dari sisi etika, seluruh sumber bersifat publik, sistem diposisikan sebagai penunjang keputusan dan bukan nasihat medis atau finansial pribadi, dan keluaran pada domain berisiko tinggi tetap ditinjau manusia; penalaran yang *grounded* justru mendukung akuntabilitas karena jejak hitungnya dapat ditelusuri.

REFERENSI

- [1] M. Merrill, M. Tan, V. Gupta, T. Hartvigsen, dan T. Althoff, “Language Models Still Struggle to Zero-shot Reason about Time Series,” dalam *Findings of EMNLP*, 2024.
- [2] “MTBench: A Multimodal Time Series Benchmark for Temporal Reasoning and Question Answering,” arXiv:2503.16858, 2025.
- [3] “VeriTime: Time Series Reasoning via Process-Verifiable Thinking Data Synthesis and Scheduling for Tailored LLM Reasoning,” arXiv:2602.07830, 2026.
- [4] “SelfBudgeter: Adaptive Token Allocation for Efficient LLM Reasoning,” arXiv:2505.11274, 2025.
- [5] “TimeOmni-1: Incentivizing Complex Reasoning with Time Series in Large Language Models,” dalam *Proc. ICLR*, 2026.
- [6] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” dalam *Proc. NeurIPS*, 2022.
- [7] L. Gao et al., “PAL: Program-aided Language Models,” dalam *Proc. ICML*, 2023.
- [8] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” dalam *Proc. ICLR*, 2022.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, dan L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” dalam *Proc. NeurIPS*, 2023.
- [10] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.
- [11] Portal data publik Indonesia yang dievaluasi: Bank Indonesia PIHPS (<https://www.bi.go.id/hargapangan>), Kementerian Kesehatan RI, dan BMKG (<https://dataonline.bmkg.go.id>), diakses 15 Juli 2026.
- [12] Z. Gou et al., “ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving,” dalam *Proc. ICLR*, 2024.
- [13] L. Chen, M. Zaharia, dan J. Zou, “FrugalGPT: How to Use Large Language Models More Cheaply and Efficiently,” arXiv:2305.05176, 2023.
- [14] L. Pan et al., “Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning,” dalam *Findings of EMNLP*, 2023.
- [15] World Bank, “World Development Indicators — Indonesia,” World Bank Open Data API, tersedia di <https://api.worldbank.org/v2/country/IDN>, diakses 15 Juli 2026.
- [16] Badan Pusat Statistik, “BPS Web API,” tersedia di <https://webapi.bps.go.id>, diakses 17 Juli 2026.
- [17] Unsloth AI, “Unsloth: Efficient QLoRA Fine-tuning,” tersedia di <https://github.com/unslothai/unsloth>, diakses 2026.